5. Constante. Variabile. Expresii în C

Programarea calculatoarelor
Lăcrămioara Stoicu-Tivadar, 2007

Constante. Variabile. Expresii în C

- ✓ Instrucţiunile unui program prelucrează date
  - constante
  - variabile

Constante. Variabile. Expresii în C

- ✓ Declaraţiile prezintă variabilele care vor fi utilizate într-un program şi specifică tipul lor (eventual şi valorile lor iniţiale).
- ✓ Operatorii specifică acţiunea care trebuie făcută cu variabilele sau constantele.
- ✓ Expresiile combină variabile şi constante pentru a produce valori noi.

5.1. Constante

✓ Constantele pot fi:

- explicitate chiar prin valoarea lor (numere, şiruri de caractere);
- desemnate prin identificatori (nume simbolice)
  - standard (INT_MAX, INT_MIN.)
  - definite de utilizator

Constante definite de utilizator

- se definesc prin intermediul identificatorilor
- definirea implică asocierea la identificator, a unor valori descrise
  - explicit (numere, caractere) sau
  - care rezultă prin evaluarea unei expresii (operanzii trebuie să fie constante, iar operatorii de tipul constantei care se defineşte)
- funcţiile simple care apar în expresie se evaluează la compilare (abs).

Exemple de constante definite de utilizator

```
const      m=-3;
const      n=abs(m)-1;
```

PC_x15.C

Definiţia unei constante definite de utilizator

Asigură valori nemodificabile

const identificator <= expresie_constantă>;

const nume_functie(const <type>*<variable name> ;)

- ✓ const - cuvânt rezervat
- ✓ identificator - numele constantei
- ✓ expresie constantă:
  - constantă simplă
  - constantă structurată
  - expresie formată din operaţii şi funcţii simple aplicate numai asupra unor constante

Constante

- ✓ O constantă de tip întreg poate conţine în final una din literele L, l, (*long*) U, u (*unsigned*).
- ✓ Pentru hexa literele a,b,c…pot fi mari sau mici, după 0x sau 0X.
- ✓ Constantele lungi: 123L.
- ✓ Constantele hexazecimale pot fi urmate de un L pentru a le considera *long*.

Constante

- Constantele reale se pot scrie
  234.987e-3 sau -0.432E3
- 1.0 2. .1 2.634 .125
  2.e52.e+5 .125e-3 2.5e5 3.1E-6
- Orice constantă reală este considerată ca fiind de tipul *double.*
- E , e – acelaşi lucru
- Aceste constante sunt *double*, în afara cazului în care sunt urmate de f, F - *float* sau l, L - long double.

Constante

- ✓ Valoarea unei constante caracter este valoarea numerică a caracterului în setul de caractere al calculatorului.
  - Spre exemplu, în setul de caractere ASCII, caracterul zero, sau '0', are valoarea 48, aceasta fiind diferită de valoarea numerică 0.
  - '0' în loc de valoarea 48, face ca programul să fie independent de o valoare particulară.
  - Constantele caracter participă în operaţiile numerice la fel ca oricare alte numere, cu toate că, cel mai adesea ele sunt folosite în comparări cu alte caractere.
  - Secvenţele *escape* au asociată o valoare întreagă care poate fi memorată pe un char.

Secvenţe *escape*

Secvența	Reprezintă
\a	bell
\b	backspace
\f	form feed
\n	newline
\r	carriage return
\t	tab
\v	vertical tab
\\	backslash
\'	quote (apostrof)
\"	double quote (ghilimele)
\?	question mark (semnul întrebării)

Constante

- ✓ O constantă-şir este o secvenţă compusă din zero sau mai multe caractere între ghilimele duble
  - "Îmi place limbajul C"
  - "" - un şir vid
- ✓ " şi " sunt delimitatori.
- ✓ Aceleaşi secvenţe *escape* folosite pentru constantele caracter se aplică şi pentru şiruri;
  - \" - caracterul ghilimele duble.

Expresii constante

- ✓ O expresie constantă este o expresie care implică numai constante.
- ✓ Astfel de expresii sunt evaluate la compilare şi nu la execuţie şi pot fi folosite în orice loc în care poate să apară o constantă.

```
#define MAXLINE 1000
char line[MAXLINE+1];
```

sau

```
secunde = 60 * 60 * ore;
```

Avantajele utilizării constantelor definite de utilizator

- ✓ programul devine mai clar prin folosirea numelor simbolice care caracterizează valoarea numerică pe care o reprezintă;
- ✓ creşte gradul de generalitate al programului (modificări doar la început)
- ✓ se precizează explicit că valorile respective nu trebuie modificate.

Constante întregi cu tipuri diferite

Conform standardului ANSI C

- ✓ O constantă simplă **zecimală** va fi cuprinsă într-unul dintre tipurile de întregi din lista următoare:

int long unsigned long

care va putea conţine valoarea.

- ✓ Constantele simple **hexazecimale** vor utiliza lista următoare:

int unsigned int long unsigned long

- ✓ Dacă după constantă apare sufixul u sau U:

unsigned int unsigned long

- ✓ Dacă sufixul este l sau L:

long unsigned long

- ✓ Dacă sufixul este u saU şi l sau L poate fi doar

unsigned long

- ✓ Un compilator bun avertizează atunci când constanta este "promovată" la o lungime mai mare şi U, u sau l, L nu sunt precizate.

PC_x16.C

Evaluarea expresiilor

- ✓ Evaluarea unei expresii se face într-o anumită ordine ţinând seama de regulile de prioritate sau precedenţă.
- ✓ Gruparea operatorilor: asociativitate.
- ✓ Atribuirea de priorităţi operatorilor – precedenţă

a + b * c + d

se grupează ca

a + (b * c) + d

- ✓ Mulţi operatori în C – 15 niveluri de precedenţe

Calificatori - qualifiers

- ✓ Calificatori de tip: *const, volatile*
- ✓ Cuvintele cheie *const* şi *volatile* pot fi aplicate oricărei declaraţii – chiar şi pentru structuri, enumerare sau în situaţii în care se foloseşte *typedef*. Operaţiunea se numeşte **calificare a declaraţiei.**

```
volatile i;
volatile int j;
const long q;
const volatile unsigned long int rt_clk;      /apare în nucleul unor sisteme în timp real
struct   {
    const long int li;
    signed char sc;
}  volatile vs;
```

Calificatori

- ✓ Este permisă declararea adresei unei date care nu este o constantă folosind un *pointer* declarat printr-un calificator *const* al aceluiaşi tip.
- ✓ Se poate astfel utiliza *pointerul* pentru a vizualiza data, fără a face modificări.
- ✓ Plasarea adresei unei date declarate *const* într-un pointer nu este permisă.

PC_x17.C

Exemplul PC_x17.C

- ✓ În acest exemplu se observă că este posibilă preluarea adresei unei constante, generarea unui *pointer* către o entitate care nu este constantă, şi apoi utilizarea acestui *pointer*, dar acest mod de lucru este eronat, determinând un **comportament nedefinit.**
- ✓ Obiectele constante au rostul de a menţine valori nemodificate.

```
char c;
char *const cp = &c;
```

- ✓ cp este *pointer* către un caracter (cu sau fără *const* este acelaşi lucru)
- ✓ *const* arată că cp nu trebuie modificat, deşi entitatea către care indică poate fi modificată; *pointer*ul este constant, nu şi entitatea către care indică acesta.
- ✓ const char *cp;
- ✓ cp est un *pointer* obişnuit, care poate fi modificat, dar entitatea către care indică nu poate fi modificată.
- ✓ Ca urmare, în funcţie de caz, pointerul sau entitatea către care indică pot fi modificabile sau nu.

volatile

- ✓ Indică faptul că o variabilă poate fi modificată de către o rutină din fundal (background routine).
- ✓ Sintaxa: volatile <definiţie date> ;
- ✓ Fiecare referire la variabila respectivă va reîncărca conţinutul din memorie mai degrabă, fără să profite de faptul că în anumite situaţii există o copie a acesteia într-un registru.

volatile

- ✓ Acest calificator de tip este utilizat în principal atunci când se realizează programe în limbaj C pentru sisteme în timp real sau embedded.
- ✓ Spre exemplu, dacă scrieţi cod care controlează un echipament (hardware device) şi plasaţi valorilor necesare în registre, la adrese absolute cunoscute.

Pentru detalii despre *volatile* consultaţi fişierul volatile.doc şi fişierele co_vo1.c şi co_vo2.c

5.2. Variabile

- ✓ Orice variabilă care apare într-un program în limbaj C trebuie declarată; se face astfel legătura între numele simbolic al variabilei şi tipul său

```
int m,n;
float mat[20] [20];
double v,w;
char ch1,ch2;
```

Variabile

- ✓ O variabilă reprezintă de fapt numele simbolic al unei locaţii de memorie
- ✓ Prin declararea variabilei i se asociază locaţiei de memorie
  - un nume şi
  - un tip
- ✓ aici se va memora o valoare de tipul declarat

Declararea unei variabile

tip_var identificator;

- ✓ identificator - poate fi unul singur sau o listă de identificatori despărţiţi prin virgulă, dacă există mai multe variabile de acelaşi tip
- ✓ tip_var - indicarea tipului variabilei

Variabile

- Când se declară o variabilă, i se poate asigna o valoare.

- Variabilele întregi sunt folosite pentru a memora numere întregi.
  int contor;
  int nr_studenti = 165;

- Variabilele reale, referite şi ca variabile *float*, stochează numere exprimate în virgulă flotantă (*floating point numbers*)

  float variabila = 0.0;
  float numar_real = 1234567.89;
- Variabilele caracter stochează valorile unui caracter

  char prima_litera = 'A';
  char o_litera= 'X';

5.3. Evaluarea expresiilor în C

- ✓ O expresie implică efectuarea unor operaţii asupra unor operanzi (constante, variabile, apeluri de funcţii), iar rezultatul evaluărilor este o valoare.
- ✓ Tipul acestei valori este **tipul expresiei.** Acesta poate fi determinat din aproape în aproape urmărind tipul rezultatului fiecărei operaţii în ordinea de evaluare a expresiei.

Expresii

- ✓ Expresiile în C sunt construite combinând operatori şi operanzi.
- ✓ x = a+b*(-c)
  - operatorii sunt =, + * and -
  - operanzii sunt variabilele x, a, b,c.
- ✓ Parantezele sunt folosite pentru a grupa subexpresii.
- ✓ Operatorii sunt binari (cu doi operanzi) sau unari (cu un operand).
- ✓ Tipurile precizate prin cuvintele *int* şi *float* sunt tipuri aritmetice; tipul *pointer* e altceva.

PC_18.C

Expresiile în C se pot construi astfel

E = expresie simplă <operatori relaţionali expresie simplă>

- ✓ *operatorii relaţionali*: <, <=, >, >=, ==,!=
- ✓ o *expresie simplă* se compune din:
  - termeni
  - operatori aditivi (+, -, OR, XOR)
- ✓ un termen conţine
  - factori
  - operatori multiplicativi (*,/,%,and,<<,>>)

Un factor poate fi

- ✓ constantă (simplă, fără semn)
- ✓ variabilă (reprezentată prin valoare)
- ✓ & urmat de o variabilă
- ✓ NOT alt factor
- ✓ + sau - alt factor
- ✓ apel de funcţie
- ✓ conversie de tip
- ✓ o expresie între paranteze

Operatorii

- ✓ Multiplicativi: *, /, %
  - Operaţiile aritmetice de înmulţire şi împărţire asupra numerelor reale şi întregi; împărţirea a doi întregi produce un rezultat trunchiat (către zero), iar restul împărţirii se referă doar la numere întregi.
  - Dacă împărţirea nu este exactă şi nici unul dintre cei doi operatori nu este negativ rezultatul împărţirii (/) este pozitiv şi rotunjit către zero; pentru a se obţine restul se foloseşte %.

9/2 = 4 şi 9%2 = 1

  - Dacă unul dintre operatori este negativ, rezultatul împărţirii (/) poate fi întregul cel mai apropiat de rezultatul real, dintr-una din părţi, şi semnul rezultatului operaţiei % poate fi pozitiv sau negativ.
  - Depinde de implementare.
  - Conversiile aritmetice obişnuite se aplică ambilor operanzi.

Operatorii

- ✓ Aditivi:+, -
  - Adunarea şi scăderea: + şi – operatori binari
  - Operatori unari: + şi – .
  - Conversiile aritmetice obişnuite se aplică ambilor operanzi ai operaţiilor aditive (pentru operatorii binari, evident).
  - Asupra operanzilor unari se aplică doar conversiile integrale.

Operatorii

- ✓ Binari: asociaţi cu operaţiile pe bit.

Operator	Efect	Conversii
&	bitwise AND	usual arithmetic conversions
|	bitwise OR	usual arithmetic conversions
^	bitwise XOR	usual arithmetic conversions
<<	left shift	integral promotions
>>	right shift	integral promotions
~	one's complement	integral promotions

PC_x19.C

Operatorii

Operatorii compuşi

- ✓ Atribuire

x=x+1 x+=1

…

Rezultatul unei atribuiri are valoarea şi tipul datei căreia i-a fost atribuit.

*=	/=	%=
+=	-=	
&=	|=	^=
>>=	<<=	

Operatorii

- ✓ Incrementare/decrementare
  - Operatori unari: -- şi ++
  - x++; ++x; x--; --x;

PC_x20.C

Operatorul &

- ✓ operator de adresare (se plasează în faţa operandului
- ✓ **&** identificator - astfel se reprezintă adresa din memorie a entităţii desemnate prin identificator

Exemplu: int x; &x -> adresa lui x

Evaluarea expresiilor

- ✓ Evaluarea unei expresii se face într-o anumită ordine ţinând seama de regulile de prioritate sau precedenţă.
- ✓ Gruparea operatorilor: asociativitate.
- ✓ Atribuirea de priorităţi operatorilor – precedenţă

a + b * c + d

se grupează ca

a + (b * c) + d

- ✓ Mulţi operatori în C – 15 niveluri de precedenţe

Precedenţa

Operatorul	Direcția de asociere	Observații
() [ ] -> .	de la stânga la dreapta	
! ~ ++ -- - + (cast) * & sizeof	de la dreapta la stânga	**unari**
*** / %**	de la stânga la dreapta	**binari**
+ -	de la stânga la dreapta	**binari**
<< >>	de la stânga la dreapta	**binari**
< <= > >=	de la stânga la dreapta	**binari**
== !=	de la stânga la dreapta	**binari**
&	de la stânga la dreapta	**binari**
^	de la stânga la dreapta	**binari**
|	de la stânga la dreapta	**binari**
&&	de la stânga la dreapta	**binari**
||	de la stânga la dreapta	**binari**
?:	de la dreapta la stânga	
= += şi restul combinațiilor	de la dreapta la stânga	**binari**
,	de la stânga la dreapta	**binari**

Precedenţa

- ✓ Regulile de precedenţă sunt important de cunoscut
  - pentru a şti cum să scrieţi expresiile care vor fi evaluate în ordinea corectă
  - pentru a citi corect expresiile scrise de altcineva.
- ✓ Prima regulă pentru o evaluare corectă este să identificaţi operatorii unari.
  - m***p se interpretează ca m înmulţit cu o expresie în care apar p şi câţiva operatori unari *.
  - a + -b++ + c → a + -(b++) + c
- ✓ Operaţiile cu aceeaşi prioritate se evaluează (ca şi în matematică) de la stânga la dreapta.

Precedenţa

- ✓ Oricât de complicată ar fi o expresie care conţine diverse tipuri, tipul subexpresiilor va fi dat de tipul operanzilor din subexpresie.

PC_x21.C

- ✓ (float)(j/i) va utiliza împărţirea în întreg şi apoi va face conversia în float.
- ✓ (float)j/i va forţa împărţirea în real.

Evaluarea expresiilor

- ✓ Expresiile logice care conţin operatorii OR şi AND pot fi evaluate:
  - complet
  - parţial (în scurt-circuit) - în fiecare subexpresie care conţine operatorul AND sau OR se evaluează primul operand şi apoi în funcţie de valoarea acestuia şi de tipul operaţiei se continuă sau nu evaluarea celui de-al doilea operand.
- ✓ Dacă primul operand al unei operaţii AND rezultă *false* atunci cel de-al doilea operand nu se mai evaluează şi rezultatul operaţiei va fi *false*
- ✓ Dacă primul operand al unei operaţii OR este *true*, atunci cel de-al doilea operand nu se mai evaluează, rezultatul operaţiei fiind *true.*

Conversia de tip - cast

- Conversie de tip - de regulă, în C se efectuează transformări asupra unor variabile dintr-o expresie.
- int i=10;
  double x=i*2;
- Compilatorul va observa că i se atribuie lui *x* o valoare de tip *int* şi va efectua automat o conversie în *double*.
- În principiu, în orice expresie în care apare un tip mai "mic" şi unul mai "mare", are loc o conversie a celui "mic" către cel "mare".

- Putem cere o conversie explicită folosind construcţia

(tip)expresie

```
double x=10.25;
int i=(int)x;
```

Conversia de tip

- ✓ Pentru a transforma valoarea unei expresii în conformitate cu un anumit tip se foloseşte conversia de tip.
- ✓ Structura sintactică a conversiei de tip include un identificator de tip urmat de o expresie între paranteze rotunde.

cast

- ✓ În cazul în care rezultatul obţinut nu are tipul dorit, atunci se poate folosi o construcţie numită *cast* (distribuire, atribuirea unui anumit rol).
- ✓ Plasarea unui nume de tip între paranteze rotunde conduce la construirea unui operator unar numit *cast*.
- ✓ Exemplu : (int)…
- ✓ Un operator *cast* face ca rezultatul evaluării expresiei din dreapta sa să aibă tipul dorit.
- ✓ Spre exemplu, dacă împărţim doi întregi, a/b, nu vom avea nici un rest. Pentru a forţa obţinerea părţii fracţionare putem folosi fie o variabilă intermediară de tip *float*, fie operatorul *cast*.
- ✓ *cast* are un efect temporar, doar asupra conţinutului variabilei şi nu asupra variabilei în sine care rămâne tot de tipul precizat în declaraţia iniţială.

cast

Declarație	Cast	Tip
int x;	(int)	int
float f;	(float)	float
char x[30];	(char [30])	array of char
int *ip;	(int *)	pointer to int
int (*f)();	(int (*)())	pointer to function returning int

Expresii - conversii

- ✓ În cazul în care sunt prevăzute conversii de întregi (short, char, operaţii pe bit, tip enumerare) se aplică următoarele reguli:
  - dacă un tip referit prin *int* poate cuprinde toate valorile tipului original, atunci valoarea este convertită la *int*;
  - în caz contrar conversia se va face la *unsigned int*.
- ✓ În acest mod se conservă atât valoarea cât şi semnul tipului original. Spre exemplu, un caracter este tratat cu semn sau fără semn în funcţie de implementare.
- ✓ Acest gen de conversii sunt frecvente, fiind parte a conversiilor aritmetice uzuale, aplicându-se operanzilor asociaţi cu operatorii unari +, -, ~ (not) sau cu shift. De asemenea se aplică şi în cazul în care o expresie este argumentul unei funcții pentru care nu a fost precizat nici un tip în cadrul prototipului.
- ✓ În cazul conversiilor între tipuri de întregi, dacă noul tip cuprinde toate valorile vechiului tip, atunci valoarea rămâne nemodificată.

Expresii - conversii

- Conversia din tipul *float* în întreg are ca efect trunchierea părţii fracţionare. Dacă tipul întreg nu poate cuprinde valoarea rămasă, atunci **comportamentul este nedefinit** – un fel de depăşire (overflow).
- Conversia *float* -> *double* -> *long double*, nu prezintă probleme, fiecare tip mai lung putând cuprinde valoarea tipului inferior şi ca urmare, nu se pierde nimic.
- În cazul în care conversia are loc invers, dacă valoarea se află în afara domeniului de cuprindere **comportamentul este nedefinit**.
- Dacă valoarea este cuprinsă în domeniu, dar nu poate fi cuprinsă exact, rezultatul va fi dat de una dintre valorile cele mai apropiate care pot fi cuprinse, în funcţie de implementare. Aceasta înseamnă că va avea loc o **pierdere de precizie**.

Expresii - conversii

- ✓ Dacă operanzii dintr-o expresie au tipuri diferite, atunci va avea loc o conversie, astfel încât să se obţină un tip comun pentru rezultat.

- ✓ Reguli pentru conversii:
  - Dacă unul dintre operanzi este *long double*, atunci celălalt este convertit la *long double* şi acesta va fi şi tipul rezultatului;
  - Dacă unul dintre operanzi este *double* atunci şi celălalt este convertit la *double* şi acesta va fi şi tipul rezultatului;
  - Dacă unul dintre operanzi este *float*, atunci celălalt este convertit la *float* şi acesta va fi şi tipul rezultatului;
  - Dacă unul dintre operanzi este *unsigned long int*, atunci celălalt este convertit la *unsigned long int* şi acesta va fi şi tipul rezultatului;
  - Dacă unul dintre operanzi este *long int*, atunci celălalt este convertit la *long int* şi acesta va fi şi tipul rezultatului;
  - Dacă unul dintre operanzi este *unsigned int*, atunci celălalt este convertit la *unsigned int* şi acesta va fi şi tipul rezultatului;
  - Dacă ambii operanzi sunt de tip întreg, *int*, atunci rezultatul este de tip întreg, *int*.